

SENAI

Relatório Técnico

React

Um Framework/Biblioteca Front-end para Construção de Interfaces Web

Atividade 02 — Desenvolvimento Web

Disciplina: Desenvolvimento de Sistemas Web

Integrantes: Ana Paula Maximo, Pedro Vitor Ribeiro Silva, Pietro Freire Rezende dos Santos, Rafaela Silva da Paz e
Tuanny Cristina Thomazelli

2026

Sumário

1. Introdução	3
2. O que é o React	3
3. Principais Características	4
3.1 Componentização	4
3.2 Virtual DOM	4
3.3 JSX (JavaScript XML)	4
3.4 Fluxo de Dados Unidirecional	4
3.5 Hooks	5
3.6 Ecossistema e Ferramentas	5
4. Vantagens do React	5
5. Desvantagens e Pontos de Atenção	6
6. Aplicações no Mercado	6
7. Exemplo Prático de Utilização em um Projeto Web	7
7.1 Estrutura do Projeto	7
7.2 Componente de Lista de Tarefas	7
7.3 Explicação do Código	8
8. Conclusão	9
9. Referências	9

1. Introdução

O desenvolvimento de interfaces para a Web evoluiu significativamente nos últimos anos, passando de páginas estáticas construídas apenas com HTML e CSS para aplicações complexas, interativas e altamente dinâmicas. Esse cenário impulsionou o surgimento de bibliotecas e frameworks Front-end que facilitam a organização, a manutenção e a escalabilidade do código no lado do cliente.

Neste relatório técnico, o grupo apresenta o **React**, uma biblioteca JavaScript amplamente utilizada para a construção de interfaces de usuário (UI). Serão abordadas suas principais características, vantagens, desvantagens, aplicações reais no mercado de trabalho e um exemplo prático de utilização em um projeto Web, com o objetivo de demonstrar, na teoria e na prática, por que essa tecnologia se tornou uma das mais relevantes do desenvolvimento Front-end atual.

2. O que é o React

O React é uma biblioteca de código aberto (open source) mantida pela Meta (antiga Facebook) em conjunto com uma grande comunidade de desenvolvedores e empresas. Lançado em 2013, o React foi criado para resolver um problema comum no desenvolvimento de interfaces: a dificuldade de manter a interface do usuário sincronizada com o estado da aplicação de forma eficiente, especialmente em sistemas com muitas atualizações dinâmicas de tela.

Tecnicamente, o React é classificado como uma **biblioteca** (library) e não como um framework completo, pois ele se concentra exclusivamente na camada de visualização (a letra "V" do padrão MVC). Entretanto, é muito comum — inclusive no meio acadêmico e profissional — que o React seja tratado como "framework Front-end", já que, ao ser combinado com outras bibliotecas do seu ecossistema (como React Router, Redux, Zustand, Axios etc.), ele forma um ambiente completo para a criação de aplicações Web robustas, conhecidas como SPA (*Single Page Applications*).

3. Principais Características

3.1 Componentização

O React organiza a interface em pequenas partes independentes e reutilizáveis chamadas **componentes**. Cada componente encapsula sua própria estrutura (HTML/JSSX), estilo e comportamento (lógica em JavaScript), podendo ser reaproveitado em diversas partes da aplicação. Essa abordagem favorece a organização do código, facilita a manutenção e permite que equipes trabalhem em paralelo em diferentes componentes sem grandes conflitos.

3.2 Virtual DOM

Uma das características mais conhecidas do React é o uso do **Virtual DOM**, uma representação leve do DOM real mantida em memória. Sempre que o estado de um componente muda, o React cria uma nova versão do Virtual DOM, compara-a com a versão anterior (processo chamado de *reconciliation* ou "diffing") e atualiza no DOM real apenas os elementos que efetivamente mudaram. Isso reduz manipulações desnecessárias do DOM, que são operações custosas para o navegador, resultando em interfaces mais rápidas e responsivas.

3.3 JSX (JavaScript XML)

O React utiliza uma extensão de sintaxe chamada **JSX**, que permite escrever uma estrutura semelhante a HTML diretamente dentro do código JavaScript. O JSX é posteriormente convertido (via ferramentas como Babel) em chamadas de função JavaScript puras. Essa característica torna o código mais legível e intuitivo, aproximando a lógica da interface visual.

```
function Saudacao() {  
  const nome = "Turma de Desenvolvimento Web";  
  return (  

```

```

    <div>
      <h1>Ola, {nome}!</h1>
    </div>
  );
}

```

Figura 1 — Exemplo simples de sintaxe JSX combinando marcação e lógica.

3.4 Fluxo de Dados Unidirecional (One-Way Data Binding)

No React, os dados fluem em uma única direção: do componente pai para o componente filho, por meio de propriedades chamadas **props**. Esse modelo torna o comportamento da aplicação mais previsível e facilita a identificação de erros, já que sempre é possível rastrear a origem de uma informação exibida na tela.

3.5 Hooks

Introduzidos na versão 16.8, os **Hooks** são funções especiais que permitem utilizar recursos como estado (*state*) e ciclo de vida em componentes funcionais, sem a necessidade de escrever componentes baseados em classes. Os principais hooks nativos são:

- **useState** — gerencia estados locais dentro de um componente;
- **useEffect** — executa efeitos colaterais, como requisições a APIs ou manipulação de eventos;
- **useContext** — compartilha dados entre componentes sem a necessidade de passar props manualmente em cada nível;
- **useRef** — acessa diretamente elementos do DOM ou mantém valores mutáveis entre renderizações.

3.6 Ecossistema e Ferramentas

O React possui um ecossistema amplo e maduro, composto por ferramentas complementares que ampliam suas funcionalidades, tais como o **React Router** (navegação entre páginas), **Redux** e **Zustand** (gerenciamento de estado global), **Next.js** (framework para renderização no servidor e geração estática) e **React Native**, que permite reaproveitar conhecimento em React para o desenvolvimento de aplicativos móveis nativos.

4. Vantagens do React

A adoção do React em larga escala pela indústria de tecnologia está diretamente relacionada às vantagens que a biblioteca oferece tanto para desenvolvedores quanto para as empresas. Entre as principais, destacam-se:

Vantagem	Descrição
Alta performance	O uso do Virtual DOM reduz atualizações desnecessárias, tornando a aplicação mais rápida.
Reutilização de código	Componentes podem ser reaproveitados em diferentes telas e até em outros projetos.
Curva de aprendizado moderada	Por ser focado apenas na camada de visualização, é mais simples de aprender do que frameworks completos.
Comunidade ativa	Grande quantidade de documentação, tutoriais, bibliotecas e suporte da comunidade.
Mercado de trabalho aquecido	É uma das tecnologias Front-end mais requisitadas em vagas de emprego atualmente.
Flexibilidade	Pode ser integrado gradualmente a projetos existentes, sem exigir reescrita completa.
Multiplataforma	Com o React Native, o conhecimento adquirido pode ser aplicado no desenvolvimento mobile.
SEO e performance com SSR	Frameworks como o Next.js permitem renderização no servidor, melhorando indexação e velocidade de carregamento.

5. Desvantagens e Pontos de Atenção

Apesar de suas inúmeras vantagens, é importante que o grupo apresente, de forma imparcial, algumas limitações do React:

- **Apenas a camada de visualização** — o React não oferece, nativamente, soluções para roteamento, chamadas HTTP ou gerenciamento de estado global, exigindo bibliotecas adicionais;
- **Configuração inicial** — em projetos mais complexos, a configuração do ambiente (bundlers, linters, gerenciadores de estado) pode exigir conhecimento técnico adicional;
- **Atualizações frequentes** — o ecossistema evolui rapidamente, o que exige atualização constante por parte dos desenvolvedores;
- **JSX pode confundir iniciantes** — a mistura de lógica e marcação, embora prática, pode ser estranha para quem está começando no desenvolvimento Front-end.

6. Aplicações no Mercado

O React é utilizado por empresas de tecnologia de grande porte em produtos que atendem milhões de usuários diariamente. Isso demonstra sua robustez, escalabilidade e confiabilidade em ambientes de produção. Alguns exemplos conhecidos de aplicações que utilizam o React em suas interfaces incluem:

Empresa/Produto	Uso do React
Facebook e Instagram	Interfaces Web construídas e mantidas pela própria criadora do React, a Meta.
Netflix	Utiliza React na interface Web para melhorar performance de carregamento e navegação.
WhatsApp Web	Interface web construída com React, garantindo atualizações em tempo real.
Airbnb	Emprega React na construção de suas páginas dinâmicas de busca e reservas.
Uber	Utiliza React (e React Native) em painéis administrativos e em parte de seus aplicativos.
Discord	Interface Web utiliza React para gerenciar comunicação em tempo real entre usuários.

Tabela 2 — Exemplos de empresas e produtos que utilizam React em produção.

Além dessas grandes corporações, o React é amplamente adotado por startups, agências de desenvolvimento e empresas de pequeno e médio porte, principalmente na construção de *dashboards* administrativos, plataformas de e-commerce, sistemas internos corporativos (ERPs Web), aplicações SaaS (*Software as a Service*) e portais institucionais. Segundo pesquisas anuais realizadas com desenvolvedores (como a Stack Overflow Developer Survey), o React figura constantemente entre as tecnologias Front-end mais utilizadas e mais desejadas pelo mercado, o que reforça sua relevância para profissionais em formação na área de desenvolvimento Web.

7. Exemplo Prático de Utilização em um Projeto Web

Para demonstrar a aplicação prática dos conceitos apresentados, o grupo desenvolveu um exemplo simples de componente React: uma **Lista de Tarefas (To-Do List)**, funcionalidade comum em projetos Web e que ilustra bem o uso de componentização, estado (state) e eventos.

7.1 Estrutura do Projeto

Um projeto React criado com ferramentas modernas (como Vite) costuma seguir a seguinte organização de pastas:

```
meu-projeto-react/  
├── node_modules/
```

```

├── public/
├── src/
│   ├── components/
│   │   └── ListaTarefas.jsx
│   ├── App.jsx
│   └── main.jsx
├── index.html
├── package.json
└── vite.config.js

```

Figura 2 — Estrutura de pastas típica de um projeto React criado com Vite.

7.2 Componente de Lista de Tarefas (To-Do List)

Arquivo: src/components/ListaTarefas.jsx

```

import { useState } from 'react';

function ListaTarefas() {
  const [tarefas, setTarefas] = useState([]);
  const [novaTarefa, setNovaTarefa] = useState('');

  function adicionarTarefa() {
    if (novaTarefa.trim() === '') return;
    setTarefas([...tarefas, novaTarefa]);
    setNovaTarefa('');
  }

  function removerTarefa(index) {
    const novaLista = tarefas.filter((_, i) => i !== index);
    setTarefas(novaLista);
  }

  return (
    <div className="lista-tarefas">
      <h2>Minhas Tarefas</h2>

      <input
        type="text"
        value={novaTarefa}
        placeholder="Digite uma nova tarefa"
        onChange={(e) => setNovaTarefa(e.target.value)}
      />
      <button onClick={adicionarTarefa}>Adicionar</button>

      <ul>
        {tarefas.map((tarefa, index) => (
          <li key={index}>
            {tarefa}
            <button onClick={() => removerTarefa(index)}>Remover</button>
          </li>
        ))}
      </ul>
    </div>
  );
}

export default ListaTarefas;

```

Figura 3 — Componente funcional ListaTarefas utilizando o hook useState.

Arquivo: src/App.jsx

```

import ListaTarefas from './components/ListaTarefas';

function App() {
  return (
    <div className="App">
      <ListaTarefas />
    </div>
  );
}

```

```
export default App;
```

Figura 4 — Componente principal `App.jsx` que renderiza o componente `ListaTarefas`.

7.3 Explicação do Código

O exemplo apresentado demonstra, de forma prática, os principais conceitos do React descritos anteriormente neste relatório:

- **useState:** o hook `useState` é utilizado para armazenar tanto a lista de tarefas quanto o valor digitado no campo de texto, permitindo que o componente "lembre" desses valores entre renderizações;
- **Componentização:** a funcionalidade foi isolada em um componente próprio (`ListaTarefas`), que é importado e utilizado dentro do componente principal (`App`), facilitando reuso e manutenção;
- **Eventos:** os eventos `onChange` e `onClick` conectam as ações do usuário (digitar, clicar) às funções que atualizam o estado da aplicação;
- **Renderização dinâmica:** o método `map()` percorre o array de tarefas e gera dinamicamente um item de lista (`<li>`) para cada tarefa cadastrada, demonstrando como o React atualiza a interface automaticamente sempre que o estado é alterado, graças ao Virtual DOM;
- **Fluxo de dados unidirecional:** o estado é sempre controlado pelo componente `ListaTarefas`, e qualquer mudança na tela é resultado direto de uma alteração nesse estado, tornando o comportamento da aplicação previsível.

Esse exemplo, embora simples, representa a base de funcionalidades presentes em sistemas reais, como painéis de gerenciamento de tarefas, carrinhos de compra em e-commerces e formulários dinâmicos, evidenciando como os conceitos do React podem ser aplicados diretamente em projetos Web utilizados no mercado.

8. Conclusão

A partir do estudo realizado, o grupo pôde compreender que o React se consolidou como uma das principais tecnologias para o desenvolvimento de interfaces Web modernas, graças a características como a componentização, o uso do Virtual DOM, o suporte a Hooks e um ecossistema amplo e ativo. Suas vantagens, como performance, reutilização de código e forte presença no mercado de trabalho, justificam sua ampla adoção por empresas de diversos portes, desde grandes corporações de tecnologia até pequenas startups.

O exemplo prático desenvolvido — uma lista de tarefas — permitiu visualizar, na prática, como os conceitos teóricos estudados se aplicam na construção de funcionalidades reais, reforçando a importância do React como ferramenta de estudo dentro da disciplina de Desenvolvimento Web. Assim, conclui-se que o domínio dessa biblioteca representa um diferencial relevante para a formação técnica e profissional dos integrantes do grupo.

9. Referências

REACT. Documentação oficial do React. Disponível em: <https://react.dev>. Acesso em: ago. 2026.

META OPEN SOURCE. React GitHub Repository. Disponível em: <https://github.com/facebook/react>. Acesso em: ago. 2026.

STACK OVERFLOW. Stack Overflow Developer Survey. Disponível em: <https://survey.stackoverflow.co>. Acesso em: ago. 2026.

VITE. Documentação oficial do Vite. Disponível em: <https://vitejs.dev>. Acesso em: ago. 2026.